

OrderService.java

```
package com.harisudhan.harisudhanmart.service;

import com.harisudhan.harisudhanmart.dao.CartDAO;
import com.harisudhan.harisudhanmart.dao.OrderDAO;
import com.harisudhan.harisudhanmart.dao.ProductDAO;
import com.harisudhan.harisudhanmart.exception.NotFoundException;
import com.harisudhan.harisudhanmart.exception.ValidationException;
import com.harisudhan.harisudhanmart.model.CartItem;
import com.harisudhan.harisudhanmart.model.Order;
import com.harisudhan.harisudhanmart.model.OrderItem;
import com.harisudhan.harisudhanmart.model.Product;
import java.math.BigDecimal;
import java.sql.Connection;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;
import javax.sql.DataSource;

/**
 * Owns the checkout transaction boundary: cart -> order + order_items + stock decrement,
 * all committed or rolled back together.
 */
public class OrderService {

    private final DataSource dataSource;
    private final OrderDAO orderDAO;
    private final CartDAO cartDAO;
    private final ProductDAO productDAO;

    public OrderService(DataSource dataSource, OrderDAO orderDAO, CartDAO cartDAO, ProductDAO productDAO) {
        this.dataSource = dataSource;
        this.orderDAO = orderDAO;
        this.cartDAO = cartDAO;
        this.productDAO = productDAO;
    }

    /** F5: place an order from the buyer's current cart via a mock payment confirmation. */
    public Order checkout(long buyerId, boolean mockPaymentConfirmed) throws ValidationException, SQLException {
        if (!mockPaymentConfirmed) {
            throw new ValidationException("payment", "Mock payment confirmation is required to place an order");
        }
        List<CartItem> cartItems = cartDAO.findByUser(buyerId);
        if (cartItems.isEmpty()) {
            throw new ValidationException("cart", "Cart is empty");
        }

        List<OrderItem> orderItems = new ArrayList<>();
        BigDecimal total = BigDecimal.ZERO;
        for (CartItem ci : cartItems) {
            Product product = productDAO.findById(ci.getProductId()).orElse(null);
            if (product == null) {
                continue;
            }
            if (product.getStockQty() < ci.getQuantity()) {
                throw new ValidationException("stock", "Insufficient stock for " + product.getName());
            }
            OrderItem oi = new OrderItem();
            oi.setProductId(product.getId());
            oi.setQuantity(ci.getQuantity());
            oi.setUnitPrice(product.getPrice());
            orderItems.add(oi);
            total = total.add(product.getPrice().multiply(BigDecimal.valueOf(ci.getQuantity())));
        }
    }
}
```

```

    Order order = new Order();
    order.setBuyerId(buyerId);
    order.setStatus(Order.Status.CONFIRMED);
    order.setTotalAmount(total);

    try (Connection conn = dataSource.getConnection()) {
        conn.setAutoCommit(false);
        try {
            Order created = orderDAO.createOrder(conn, order, orderItems);
            conn.commit();
            cartDAO.clear(buyerId);
            return created;
        } catch (SQLException e) {
            conn.rollback();
            throw e;
        } finally {
            conn.setAutoCommit(true);
        }
    }
}

public List<Order> buyerHistory(long buyerId) throws SQLException {
    return orderDAO.findByBuyer(buyerId);
}

public List<Order> sellerIncomingOrders(long sellerId) throws SQLException {
    return orderDAO.findBySellerProducts(sellerId);
}

public List<OrderItem> itemsFor(long orderId) throws SQLException {
    return orderDAO.findItems(orderId);
}

public Order findByIdOrThrow(long orderId) throws NotFoundException, SQLException {
    return orderDAO.findById(orderId).orElseThrow(() -> new NotFoundException("Order not found"));
}

/** 02: Pending -> Confirmed -> Shipped -> Delivered workflow, admin/seller driven. */
public void updateStatus(long orderId, Order.Status status) throws SQLException {
    orderDAO.updateStatus(orderId, status.name());
}
}

```